

LLMProvider

LLMProvider

1つのインターフェース、43のプロバイダー — サーキットブレーカー、リトライ、ヘルスチェックを標準装備。

概要

LLMProviderは、汎用的で再利用可能なGoモジュールであり、統一されたLLMProviderインターフェースと、その周辺に実装されたプロダクションレベルの耐障害性パターン（サーキットブレーカー、ヘルスモニタリング、バックオフ付きリトライ、遅延ロード）を定義します。さらに、この単一の契約のもとに43の具体的なプロバイダー実装を提供し、OpenAI互換の汎用アダプターと、ハードコードされたフォールバックを排除した正直なモデルディスカバリー機能を備えています。

短い説明

再利用可能なGoモジュール。LLMProviderインターフェース（Complete、CompleteStream、HealthCheck、GetCapabilities、ValidateConfig）と、耐障害性の基本機能（サーキットブレーカー、ヘルスモニター、ジッターバックオフ付きリトライ、遅延初期化）を43のプロバイダーアダプターとOpenAI互換の汎用アダプターを通じて提供。スレッドセーフ。

長い説明

LLMProviderは、LLMを利用するあらゆるサービスが必要としながらも、ほとんどの場合適切に構築されていない抽象化レイヤーです。これは、デモと実際のトラフィックに耐えうるシステムを分ける、地味ながら重要な基盤部分です。このモジュールは、単一の機能を意識したインターフェース（Complete、CompleteStream、HealthCheck、GetCapabilities、ValidateConfig）を定義し、アプリケーションコードが43のバックエンドのどれを呼び出す場合でも、常に同じ契約に基づいて動作するようにします。さらに、脆弱なプロバイダー呼び出しをプロダクション環境で安心して運用できるようにするための運用強化機能を提供します。

三状態のサーキットブレーカー（クローズ → オープン → ハーフオープン）は、あらゆるプロバイダー（ストリーミングチャネルを含む）を透過的にラップし、空のストリームも正しく障害としてカウントします。これにより、単一の不調なバックエンドがサーキットブレーカーをトリップさせ、サービス全体をダウンさせることを防ぎます。中央のCircuitBreakerManagerがすべてのブレーカーを一元管理します。

設定可能なヘルスマニターは、プロバイダーの状態を「正常」「劣化」「異常」「不明」に連続的に分類し、閾値と間隔に基づくチェックを行うことで、障害が発生してから初めて気づくのではなく、劣化を事前に検知します。

リトライロジックは、指数バックオフにジッターを組み合わせ、状況に応じた判断を行います。リトライすべきエラー（429、5xx、一時的なネットワーク障害）にはリトライを試み、4xxエラーやキャンセルされたコンテキストには無駄なリトライを行いません。また、遅延時間を制限することで、バックオフの暴走を防ぎます。

遅延初期化パターンは、各プロバイダーの構築を最初の実際の使用まで遅らせる設計となっており、43のプロバイダーすべてを登録しても、実質的なコストは発生しません。

このモジュールには、43の具体的なプロバイダーパッケージと、genericというOpenAI互換の汎用アダプターが含まれています。このアダプターは、/v1/chat/completionsエンドポイントに対して完全なインターフェースを実装し、ベアラー認証やSSEストリーミング（正しい[DONE]ハンドリングを含む）に対応しています。そのため、専用パッケージがないベンダーでも、アダプターをURLに向けるだけで、第一級の市民として扱われます。

認証情報は、apikeys（厳格なApiKey_<Provider>の命名規則に従う）の1か所でのみ解決され、テストスイートではハードコードされたキーが通過しても、実際のキーが正しく接続されておらず、本番環境で障害が発生するという類のバグを根本から排除します。

モデルディスカバリーは、意図的に、そしてほとんど頑固なほど正直に設計されています。TTLキャッシュの背後で実際のプロバイダーAPIに問い合わせを行い、ガバナンスに基づき、古いハードコードされたフォールバック階層は完全に削除されました。ライブディスカバリーが失敗した場合、LLMProviderは古いカタログではなく「何も返さない」ため、呼び出し側が一見有効に見えるモデルIDを受け取っても、実際には呼び出せないという事態を防ぎます。

これらすべての機能は、並行使用に対応したスレッドセーフな設計となっています。

コンテンツ

なぜこれを作ったのか

LLMの素朴な呼び出しは本番環境で失敗する——プロバイダーはレート制限をかけたり、性能を低下させたり、ダウンしたりする。そして、一つの不調なバックエンドがサービス全体を道連れにする。モデルカタログは変動し、ハードコードされたリストはもはや機能しないIDを呼び出し側に渡してしまう。LLMProviderはインターフェース、耐障害性パターン、そして信頼性の高いディスカバリーを一元化し、すべてのコンシューマーが耐障害性と真正性を無償で継承できるようにする。

なぜゲームチェンジャーなのか

「LLMプロバイダーを統合する」という作業は、たった一つの動作に集約される——一つのインターフェースを実装するか、あるいは汎用アダプターをエンドポイントに向けるだけで、そのプロバイダーは自動的かつ透過的にサーキットブレーカー、ヘルスマonitoring、ジッターバックオフリトライでラップされる。耐障害性は、各チームが（不十分に、締め切りに追われて、最初の障害発生後に）再発明するものではなく、43のバックエンドすべてにわたってライブラリのデフォルト動作となる。信頼性エンジニアリングは一度書かれ、厳しくテストされ、それをインポートするすべての人が無償で継承する。

革新的なポイント

- 一つの機能認識型インターフェース——コンプリーション、ストリーミング、ヘルス、機能、設定バリデーションが、すべてのバックエンドが同一に遵守する単一の契約に集約される。
- 透過的なサーキットブレーカーのラッピング——ストリームも含む。ブレーカーはCompleteStreamのチャンネルを保護し、単なるリクエスト/レスポンスだけでなく、空のストリームもそれが実際に失敗であると扱う。デッドロックを回避し、ロック外でリスナーに通知する。
- 43のプロバイダーパッケージ + 汎用**OpenAI**互換アダプター——専用パッケージはシンプルに保たれ、未登録のベンダーでも/v1/chat/completionsに対応していれば、アダプターを向けるだけで即座に利用可能。
- 単一の認証情報管理 (**apikeys**) ——ApiKey_<Provider>環境変数を読み取る場所は厳密に一つだけ。これにより、「テストは通るが本番は壊れる」というミスマッチを構造的に排除し、単なる警告にとどまらない。
- 信頼性の高いモデルディスカバリー（ハードコードされたフォールバックなし）——TTLキャッシュの背後にあるライブプロバイダーAPI。障害時にはnilを返し、決して古くなったり捏造されたカタログを渡して呼び出せないIDを提供しない。
- **sync.Once**による遅延初期化——構築は最初の使用まで遅延され、43のプロバイダーすべてを登録しても、実際に呼び出すまではほとんどコストがかからない。
- グラフ防止・多地域対応の**Challenge**スタック——サーキット、ヘルス、リトライの動作を5つの地域で実際に実行するランナー。ペアリングされたミューテーションテストで制御され（非ミューテーション時は

終了コード0、ミューテーション注入時は終了コード99を強制）、テストが通れば動作が保証される。

最大の技術的課題とその解決方法

- プロバイダーの連鎖的障害。一つの不安定なバックエンドがサービス全体をダウンさせてはならない。解決策は、三状態のサーキットブレーカー（クローズ→オープン→ハーフオープン）で、あらゆるプロバイダーとそのストリームを透過的にラップ。持続的な障害でオープン状態に移行し、ハーフオープン状態で回復を試みる。中央のCircuitBreakerManagerが調整。
- 一時的なエラーとレート制限。ステータス認識型の指数バックオフ + ジッターで解決—— $\min(\text{InitialDelay} \cdot \text{Multiplier}^{(n-1)}, \text{MaxDelay}) \pm \text{jitter}$ により、リトライが同期してサンダーハード現象を引き起こさない。リトライすべきもの（429、500、502、503、504、ネットワークエラー）のみをリトライし、キャンセルされたコンテキストやその他の4xxには無駄な試行を行わない。
- 多数の登録済みだが未使用のプロバイダーのスケールリング。43のプロバイダーが登録されていても、実際に使用されるのはごく一部。積極的な構築は無駄でしかない。解決策はsync.Onceによる遅延初期化で、実際に呼び出されたプロバイダーだけがセットアップコストを負担する。
- 無効なモデルIDの提供。解決策は、ハードコードされたディスカバリーフォールバック層を完全に削除（CONST-036に基づく）し、ライブディスカバリーの障害時には何も返さない。さらに、防御的なコピーオンリターンにより、呼び出し側がキャッシュを変更したり、他の読み取りと競合したりしないようにする。真正性は構造的に強制される。
- ストリーミング + 並行処理の正確性。微妙な障害モードは、ブレーカーのロックとリスナーコールバック間のデッドロック。解決策は、リスナーをスナップショットし、ロック外で5秒のタイムアウト付きで通知。リセット時にはロックを解除してから通知。すべてのコンポーネントは並行使用を前提に設計され、-raceスイートで検証済み。

コンテンツ

テックスタック

- **Go (1.25.3)** — 一流の並行処理、スタティックバイナリ、強力な標準ライブラリを備え、モジュール本体、インターフェース、すべての耐障害性プリミティブ、そして43のアダプターを包括的に提供。
- **net/http**（標準ライブラリ） — 依存関係を排除したHTTP実装：プロバイダーごとのクライアント、汎用OpenAI互換アダプター、ライブディスカバリーコールを支え、サードパーティのトランスポートを監査・修正する必要はない。
- **logrus** — 構造化され、レベルに応じたロギングを、オペレーターが可視性を必要とする箇所（サーキットブレーカーの状態遷移やディスカバリーパス内）に正確に配置。
- **testify** — テストスイートを駆動し、特に突然変異ブランチのピン留め機能により、グリーンランが真に意味を持つように設計。

- **yaml.v3** — i18nバンドルや設定ファイルを、人間が編集可能な形式で解析。
- **digital.vasic.models** — 共有のLLMRequest/LLMResponse/ProviderCapabilities型を一元管理し、すべてのアダプターが同じ語彙で通信できるように（ドキュメント化されたランタイム依存関係）。
- ファーストパーティパッケージ群 — circuit、health、retry、apikeys、discovery、providers/（43のベンダー + generic）、およびi18n: 耐障害性と統合面を、小さな独立してテスト可能な単位に分割し、モノリシックな構造を回避。
- **.env + ~/api_keys.sh (ApiKey_<Provider>規約)** — 資格情報の唯一の真正な情報源として機能し、テスト環境と本番環境で同じ方法でキーを設定可能に。
- **Makefile**レーススイート (**-race -p 1**) + チャレンジランナー — ブラフ防止の要: レース検出器が並行処理の正確性を証明し、チャレンジランナーが実際の挙動を混乱、DDoS、スケーリング、ストレス、ライブディスカバリー、無停止シナリオを通じて徹底検証。

ステータスと正直な注意点

- ステータス: ベータ版。分離可能な再利用モジュール。GitHubリポジトリは公開済み。
- ライセンス: 未定。不整合あり — doc.goはMITと記載されているが、Apache-2.0形式のLICENSEファイルが存在。公開前に確認が必要。
- LLMsVerifierは、標準モデルカタログの上流単一情報源。helix-deps.yamlマニフェストは古い状態 (deps: [])と宣言されているが、ドキュメントではdigital.vasic.modelsへの依存が明記)。ディスカバリーの「Tier 2 (models.dev)」は計画段階のスタブであり、現時点では有効ではない。

優先度レベル: Helix-プライマリ (LLM-インフラストラクチャクラスター — 分離可能な再利用モジュール)。HelixTrackに次ぐ優先順位。